

การทำงานแบบลำดับ (sequential composition)

หลักคือ:

เวลารวม = บวกกัน แล้ว “เลือกพจน์ที่โตเร็วที่สุด” เป็นผลลัพธ์

ดังนั้นขอยกตัวอย่างอัลกอริทึม 4 กรณี ตามแต่ละช่องในสไลด์

$$\textcircled{1} \Theta(n) + \Theta(n) \Rightarrow \Theta(n)$$

ตัวอย่าง

```
for(int i = 0; i < n; i++)  
    sum += a[i];
```

} n round

```
for(int i = 0; i < n; i++)  
    if(a[i] > mx) mx = a[i];
```

} n round

$$\textcircled{2} \Theta(n) + O(n) \Rightarrow \Theta(n)$$

ตัวอย่าง

```
for(int i = 0; i < n; i++)  
    cout << i; -1
```

} n round

```
for(int i = 0; i < n; i++)  
    if(a[i] == key) break; -1
```

} n round

③ $\Theta(n) + O(n^2) \Rightarrow O(n^2)$

ตัวอย่าง

```
for(int i = 0; i < n; i++)
    cout << i; — 1

for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        if(a[i][j] == key) break;
```

④ $\Theta(n^2) + O(n) \Rightarrow \Theta(n^2)$

ตัวอย่าง

```
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        sum += a[i][j];

for(int i = 0; i < n; i++)
    if(b[i] == key) break;
```

สรุปหลักสำคัญ

เมื่อเป็น การทำงานแบบลำดับ

$$T(n) = T_1(n) + T_2(n)$$

ให้:

1. บวกพจน์ก่อน
2. เลือกพจน์ที่โตเร็วที่สุด
3. ตัดค่าคงที่ทิ้ง

if...then...else (การเลือกทาง)

หลักคิดคือ:

เวลา = เวลาตรวจเงื่อนไข + max(เวลาของแต่ละ branch)

เพราะใน worst-case ต้องพิจารณาทางที่ช้าที่สุด

ตัวอย่าง

```
bool flag = false;
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        if(a[i] == a[j] && i != j)
            flag = true;

if(flag) {
    sort(a, a+n);  $\longrightarrow$   $best = n \log n$ 
}
else {
    for(int i = 0; i < n; i++)
        cout << a[i];
}
```

วิเคราะห์เวลา

$$T(n) = \Theta(n^2) + \max(O(n \log n), \Theta(n))$$

แต่ n^2 โตเร็วกว่า $n \log n$ และ n

$\Rightarrow \Theta(n^2)$

```

int sum = 0;
for(int i = 0; i < n; i++)
    sum += a[i];

if(sum > 0) {
    sort(a, a+n);
}
else {
    for(int i = 0; i < n; i++)
        cout << a[i];
}

```

วิเคราะห์เวลา

$$T(n) = \Theta(n) + \max(O(n \log n), \Theta(n))$$

และ $n \log n$ โตเร็วกว่า n

$$\Rightarrow O(n \log n)$$

(เขียน Θ ไม่ได้ เพราะ branch ใหญ่สุดเป็น $O(n \log n)$ ไม่ใช่ Θ)

สรุปกฎสำหรับ if-else

$$T(n) = T_{condition}(n) + \max(T_{true}(n), T_{false}(n))$$

ให้

1. วิเคราะห์เวลาเงื่อนไขก่อน
2. วิเคราะห์แต่ละ branch
3. เลือก branch ที่โตเร็วที่สุด
4. บวกกับเงื่อนไข
5. ตัดพจน์เล็กทิ้ง

```

hanoi(n, a, b, c){
    if(n == 0) return;

    hanoi(n-1, a, b, c);           // เรียกครั้งที่ 1
    print("move", n, a, "->", c);  // งานคงที่
    hanoi(n-1, b, a, c);           // เรียกครั้งที่ 2
}

```

วิเคราะห์เวลา

ให้ $T(n)$ = เวลาการทำงานของ hanoi(n)

จากโค้ด:

- เรียกตัวเอง 2 ครั้ง ขนาด (n-1)
- มีงานคงที่ 1 ครั้ง (print)

ดังนั้น

$$T(n) = 2T(n - 1) + \Theta(1)$$

แก้ recurrence

คลี่ออก:

$$\begin{aligned}
 T(n) &= 2T(n - 1) + 1 \\
 &= 2(2T(n - 2) + 1) + 1 \\
 &= 4T(n - 2) + 2 + 1 \\
 &= 8T(n - 3) + 4 + 2 + 1
 \end{aligned}$$

จะได้รูปแบบ:

$$T(n) = 2^n T(0) + (2^n - 1)$$

เพราะ $T(0) = \Theta(1)$

ดังนั้น

$$T(n) = \Theta(2^n)$$

รูปแบบ Master Theorem:

$$T(n) = aT(n/b) + f(n)$$

เปรียบเทียบ $f(n)$ กับ $n^{\log_b a}$

1. $T(n) = T(n/2) + 1$

- $a = 1$
- $b = 2$
- $f(n) = 1$
- $n^{\log_2 1} = n^0 = 1$

ดังนั้น

$$T(n) = \Theta(\log n)$$

2. $T(n) = 2T(n/2) + 1$

- $a = 2$
- $b = 2$
- $f(n) = 1$
- $n^{\log_2 2} = n^1 = n$

ดังนั้น

$$T(n) = \Theta(n)$$

3. $T(n) = 2T(n/2) + n^2$

- $a = 2$
- $b = 2$
- $f(n) = n^2$
- $n^{\log_2 2} = n$

n^2 โตเร็วกว่า n

ดังนั้น

$$T(n) = \Theta(n^2)$$

4. $T(n) = 2T(n/2) + n$

- $a = 2$
- $b = 2$
- $f(n) = n$
- $n^{\log_2 2} = n$

ดังนั้น

$$T(n) = \Theta(n \log n)$$

5. $T(n) = 2T(n/3) + n$

- $a = 2$
- $b = 3$
- $f(n) = n$
- $n^{\log_3 2} = n^{0.63}$ ($\log_3 2 \approx 0.63$)

n โตเร็วกว่า

$$T(n) = \Theta(n)$$

แบบฝึกหัดเพิ่มเติม

Part A: Basic Loop

1.

```
for(int i=0;i<n;i++)  
  
    cout << i;
```

2.

```
for(int i=1;i<n;i*=2)  
  
    cout << i;
```

3.

```
for(int i=0;i<n;i++)  
  
    for(int j=0;j<n;j++)  
  
        cout << i+j;
```

4.

```
for(int i=0;i<n;i++)  
  
    for(int j=0;j<i;j++)  
  
        cout << i+j;
```


Part B: If-Else + Loop

5.

```
if(n%2==0)
```

```
    for(int i=0;i<n;i++)
```

```
        cout << i;
```

```
else
```

```
    for(int i=0;i<n*n;i++)
```

```
        cout << i;
```

6.

```
for(int i=0;i<n;i++)
```

```
    if(a[i]==key) break;
```

Part C: Mixed Growth Rate

7.

```
for(int i=0;i<n;i++)
```

```
    for(int j=1;j<n;j*=2)
```

```
        cout << i+j;
```

8.

```
for(int i=1;i<n;i*=2)
```

```
    for(int j=1;j<n;j*=2)
```

```
        cout << i+j;
```

Part D: Recurrence (Master Method)

9.

$$T(n) = 3T(n/2) + n$$

10.

$$T(n) = T(n/2) + n$$

11.

$$T(n) = 4T(n/2) + n^2$$

12.

$$T(n) = 3T(n/4) + n \log n$$

Part E: Challenge

13.

```
for(int i=0;i<n;i++)
```

```
    for(int j=0;j<n;j++)
```

```
        for(int k=0;k<n;k++)
```

```
            cout << i+j+k;
```

14.

```
for(int i=0;i<n;i++)  
  
    for(int j=i;j<n;j++)  
  
        for(int k=0;k<j;k++)  
  
            cout << i+j+k;
```

15.

```
void f(int n){  
  
    if(n<=1) return;  
  
    f(n/2);  
  
    f(n/2);  
  
}
```